



**TRIBHUVAN UNIVERSITY
INSTITUTE OF ENGINEERING
PURWANCHAL CAMPUS**

**A MINOR PROJECT PROPOSAL
ON
"DESIGN AND IMPLEMENTATION OF A TWO-WHEELED
SELF-BALANCING ROBOT WITH OPTIMIZED PID CONTROL"**

**BY
AADESH DAHAL(PUR080BEI002)
NISHCHAL POKHREL(PUR080BEI027)
PRASHAN CHENTA RAI(PUR080BEI030)
YOGESH KHADKA(PUR080BEI048)**

**DEPARTMENT OF ELECTRONICS AND COMPUTER ENGINEERING
PURWANCHAL CAMPUS
DHARAN, NEPAL**

JUNE 10, 2026

TABLE OF CONTENTS

LIST OF FIGURES	iii
LIST OF TABLES	iv
LIST OF ABBREVIATIONS	v
1 INTRODUCTION	1
1.1 Background	1
1.2 Gap Identification	1
1.3 Motivation	2
1.4 Objectives	2
1.5 Application	2
2 LITERATURE REVIEW	3
3 METHODOLOGY	4
3.1 Overview	4
3.2 Requirement analysis	5
3.3 Simulation and Prototyping	6
3.3.1 System Design and Hardware Implementation	6
3.4 Sensor Processing and Control Algorithm	8
3.5 Motor Control and System Workflow	8
3.6 Calibration, Testing, and Performance Evaluation	8
4 EXPECTED RESULTS	9
4.1 Expected Output	9
4.2 Budget Analysis	9

5	TIME SCHEDULE	10
	REFERENCES	11

LIST OF FIGURES

Figure 3.1: Closed-loop control block diagram of the two-wheeled self-balancing robot	4
Figure 3.2: Principle of inverted pendulum in a self-balancing robot	5

LIST OF TABLES

Table 3.1: Hardware Components and Their Use	7
Table 4.1: Project Budget Estimation	9

LIST OF ABBREVIATIONS

DC	: Direct Current
ESP32	: Espressif 32-bit Microcontroller
IMU	: Inertial Measurement Unit
Li-ion	: Lithium-ion
MPU	: Motion Processing Unit
PID	: Proportional – Integral – Derivative
PWM	: Pulse Width Modulation

CHAPTER 1

INTRODUCTION

A self-balancing robot is a device that maintains its upright position using sensors, a microcontroller, and motor control, typically modeled as a two-wheeled inverted pendulum. It operates by continuously measuring tilt angle and angular velocity using sensors like the MPU6050, with the data processed by a microcontroller. A PID controller then calculates the required motor adjustments, allowing the motors to change wheel speed and direction to keep the robot balanced under its center of gravity.

1.1 Background

The development of self-balancing robots has attracted significant interest due to their applications in transportation and automation. These two-wheeled robots are inherently unstable and require continuous control, similar to an inverted pendulum, using sensors and feedback mechanisms. Among various methods, the PID controller is widely used for its simplicity and effectiveness; however, improper tuning can cause instability and poor performance. Therefore, optimizing PID parameters is essential for improving stability, response time, and efficiency. This project focuses on designing and implementing a two-wheeled self-balancing robot with optimized PID control for reliable and stable operation.

1.2 Gap Identification

Many existing self-balancing robots rely on expensive hardware or complex control algorithms, limiting their applicability in low-cost robotic platforms and practical implementations. There is limited work on systematically optimizing PID parameters for microcontrollers such as the ESP32, which offers higher processing speed and built-in wireless communication for future enhancements. Moreover, few studies integrate a lightweight sensor fusion algorithm with a practical tuning procedure on a generalized two-wheeled platform. This project addresses these gaps by implementing an optimized PID controller on an ESP32 and combining efficient tilt estimation using an external IMU with experimental gain tuning to achieve stable and cost-effective balancing.

1.3 Motivation

The motivation behind this project is to understand and apply the basic concepts of control system and robotics in practical manner. A two-wheeled self-balancing robot provides an excellent platform to study real-time feedback control, as it requires precise and continuous adjustments to maintain stability. Additionally, improving PID controller performance through optimization helps enhance system efficiency and reliability. This project also aims to develop hands-on skills in electronics, programming and system design, while contributing to the growing demand for intelligent and autonomous robotic systems.

1.4 Objectives

- To design and implement a two-wheeled self-balancing robot using optimized PID control system to achieve stable and efficient balancing.
- To tune and optimize PID parameters for improved stability and response.

1.5 Application

Self-balancing robots have wide applications across many fields due to their stability, compact design, and efficient movement. In everyday life, they are used in personal transportation devices like Segways and for assisting elderly or disabled individuals in mobility. In industries and warehouses, they help in material handling and automation, improving efficiency and reducing manual labor. They are also useful in surveillance and security systems, as they can navigate tight spaces and monitor environments effectively. In education and research, self-balancing robots are valuable tools for studying robotics, control systems, and artificial intelligence.

CHAPTER 2

LITERATURE REVIEW

Two-wheeled self-balancing robots (TWSBRs) are underactuated, nonlinear systems that require continuous feedback control. This review focuses on the most relevant works for a low-cost, ESP32-based self-balancing robot using PID control.

A comprehensive review of control strategies for TWSBRs is presented in [1], covering classical PID, LQR, nonlinear methods, and intelligent techniques. It provides practical guidelines for selecting controllers based on computational resources and robustness. This work confirms that a well-tuned PID controller is an excellent choice for embedded real-time applications.

A rigorous evaluation of PID control for a TWSBR is given in [2], where the controller reaches dynamic balance in under 1.2 seconds. The maximum tolerable disturbance angle is quantified as $\theta = 0.0524$ rad, and stability is maintained under sudden payload changes, demonstrating robustness.

A complete ESP32-based implementation is documented in [3], using an MPU6050 IMU and PID gains $K_P = 6.0$, $K_I = 0.1$, $K_D = 1.5$. It achieves tilt variations of $\pm 2.5^\circ$ with a hardware cost of USD 127, including gyro drift compensation and dual-mode control.

Practical sensor fusion using a Kalman filter on an ESP32 is demonstrated in [4] by reverse-engineering a hoverboard. The platform autonomously balances, carries payloads, and operates with high energy efficiency, offering a sustainable low-cost solution.

An educational PID-based self-balancing robot using an ESP32 is presented in [5], detailing sensor calibration and PID tuning methods. The robot processes MPU6050 angle signals to drive DC motors for smooth, balanced motion, serving as a practical guide for implementation.

In summary, these five works provide a complete foundation: theoretical overview, validated PID design, ESP32 implementation metrics, Kalman-filter sensor fusion, and practical tuning procedures. An optimised PID controller with robust sensor fusion remains the most practical choice for this project.

CHAPTER 3

METHODOLOGY

3.1 Overview

A self-balancing robot operates as an inverted pendulum, requiring continuous feedback control to stay upright. As shown in Figure 3.1, a closed-loop system compares the desired upright setpoint with the actual state to generate an error, which is processed by a PID controller on a microcontroller. The controller outputs a PWM signal to the motor driver, adjusting DC motor movement to correct tilt. Sensor data from the accelerometer and gyroscope provides constant feedback, allowing the system to minimize error and maintain stability.

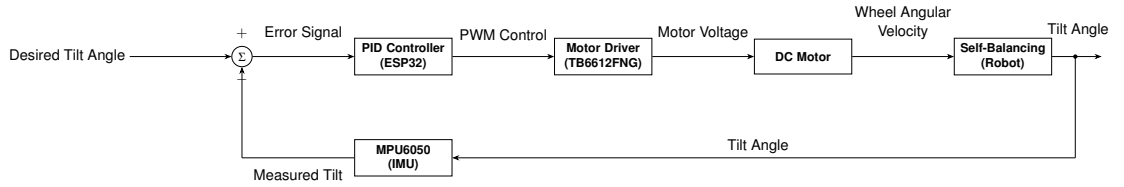


Figure 3.1: Closed-loop control block diagram of the two-wheeled self-balancing robot .

An inverted pendulum is a system where a body is balanced upright on a pivot with its center of mass above the base, making it inherently unstable and requiring continuous correction. In a self-balancing robot, the body acts as the pendulum, while the wheels move to counter tilts, using sensors and control algorithms to maintain stability through constant feedback.

The robot body acts as an inverted pendulum governed by the rotational equation of motion:

$$(I_b + mL^2)\ddot{\theta} - mgL \sin \theta = -\tau \quad (3.1)$$

where m is the mass of the robot body, L is the distance from the wheel axis to the center of mass, I_b is the moment of inertia of the body, g is gravitational acceleration, θ is the tilt angle from vertical, and τ is the corrective motor torque.

Applying the small-angle approximation ($\sin \theta \approx \theta$) near the upright equilibrium:

$$(I_b + mL^2)\ddot{\theta} - mgL\theta = -\tau \quad (3.2)$$

This shows that without corrective torque ($\tau = 0$), any small deviation θ grows exponentially, confirming the system's inherent instability. The PID controller continuously computes τ to drive $\theta \rightarrow 0$.

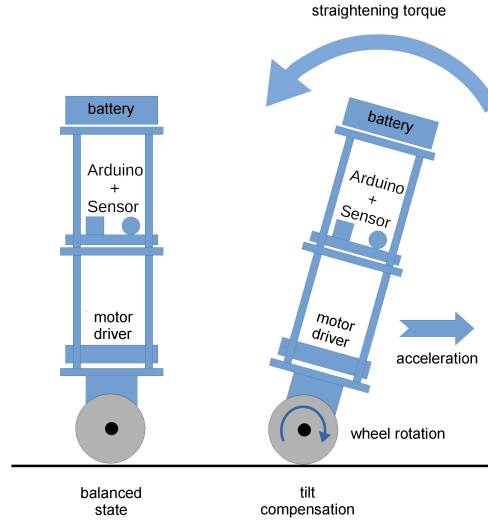


Figure 3.2: Principle of inverted pendulum in a self-balancing robot

3.2 Requirement analysis

Functional Requirements: The system must maintain balance using an inverted pendulum approach by continuously measuring tilt with an IMU and applying a PID control algorithm to generate PWM signals for real-time motor control and error correction.

Hardware Requirements: The robot requires a microcontroller (Arduino/ESP32), IMU sensor (accelerometer and gyroscope) motor driver (L298N/TB6612FNG), DC motors with wheels, and a stable power supply.

Software Requirements: The software must support real-time sensor data acquisition, PID control implementation, PWM signal generation, and parameter tuning for optimal performance.

Non-Functional Requirements: The system should ensure fast response, stability, accuracy, noise resistance, reliability, compact design, and energy efficiency during operation.

3.3 Simulation and Prototyping

Before physical implementation, the system is simulated using different simulation software to verify circuit connections, sensor interfacing, and initial PID behavior in a virtual environment. The simulation model includes the MPU6050, ESP32, TB6612FNG motor driver, and DC motors, connected as per the proposed design. Initial PID gain values for K_p , K_i , and K_d are adjusted within the simulation to observe preliminary tilt response, and the validated parameters are then used as a baseline for physical calibration described in Section 3.7.

3.3.1 System Design and Hardware Implementation

The self-balancing robot is designed as a closed-loop control system consisting of a microcontroller (Arduino/ESP32), IMU sensor (accelerometer and gyroscope), motor driver, and DC motors mounted on a two-wheel chassis. The components are interconnected such that sensor data is sent to the controller, which processes the information and drives the motors through the motor driver. Proper circuit connections and mechanical design ensure stable operation of the system. The circuit design of the proposed system is shown in Figure 3.3.

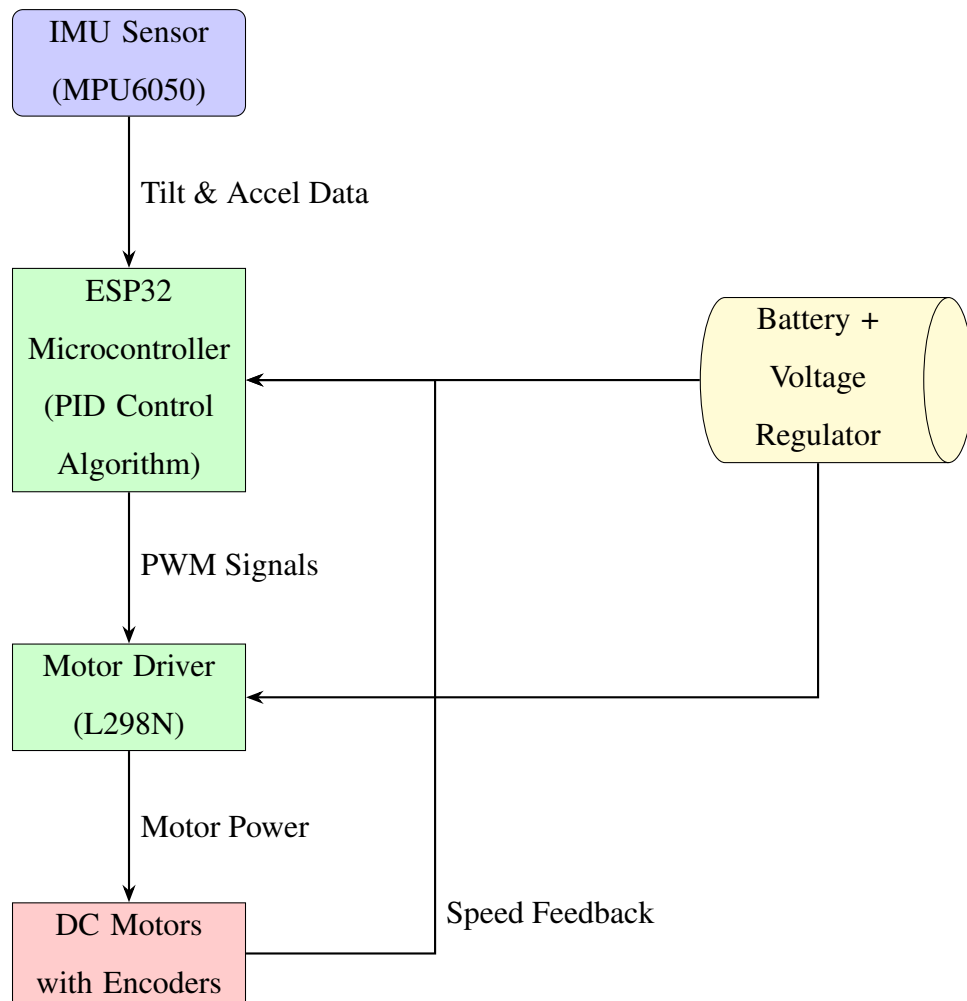


Figure 3.3: Self-Balancing Robot Internal Architecture (ESP32)

The diagram illustrates the connection between the MPU6050 sensor, microcontroller, motor driver, and DC motors, where sensor data is processed to control motor movement for maintaining balance.

Table 3.1: Hardware Components and Their Use

S.N	Component	Use
1	MPU6050 (IMU Sensor)	Measures tilt and angular motion
2	Microcontroller (ESP32/Arduino)	Processes data and implements PID control
3	Motor Driver (TB6612FNG)	Controls speed and direction of motors
4	DC Gear Motors	Provide motion for balancing
5	Li-ion Battery	Supplies power to the system
6	Buck Converter	Regulates voltage for components
7	Wheels	Enable movement of the robot
8	Chassis	Supports and holds all components
9	Jumper Wires	Provide electrical connections

3.4 Sensor Processing and Control Algorithm

The IMU sensor continuously measures the robot's tilt and motion. The raw data is processed using filtering techniques to obtain a stable angle. This angle is compared with the desired upright position to generate an error signal, which is fed into a PID controller. The controller computes corrective action based on proportional, integral, and derivative terms and generates a PWM signal to control the motors.

3.5 Motor Control and System Workflow

The PWM signal from the controller is applied to the motor driver, which regulates the speed and direction of the DC motors. The system operates in a continuous loop: reading sensor data, calculating angle and error, applying PID control, and adjusting motor motion to maintain balance.

3.6 Calibration, Testing, and Performance Evaluation

Sensor calibration and PID tuning are performed to achieve stable operation and minimize oscillations. The system is tested under different conditions to evaluate response time, accuracy, and stability, ensuring reliable performance against disturbances.

CHAPTER 4

EXPECTED RESULTS

4.1 Expected Output

The expectation of this project is that the robot will be able to maintain an upright position on two wheels without external support, demonstrating stable and smooth balancing behavior. With the use of optimized PID parameters, the robot should respond quickly to disturbances, minimize oscillations, and recover efficiently from external tilts. Overall, the project is expected to achieve improved stability, faster response time, and reliable performance compared to basic or non-optimized control systems.

When subjected to an initial tilt of 15° , the robot is expected to return to the upright equilibrium position within approximately 4–5 seconds. The response exhibits no overshoot or oscillation, indicating a well-tuned PID controller operating in the critically damped regime. The tilt angle decays smoothly toward zero, demonstrating stable closed-loop behavior.

4.2 Budget Analysis

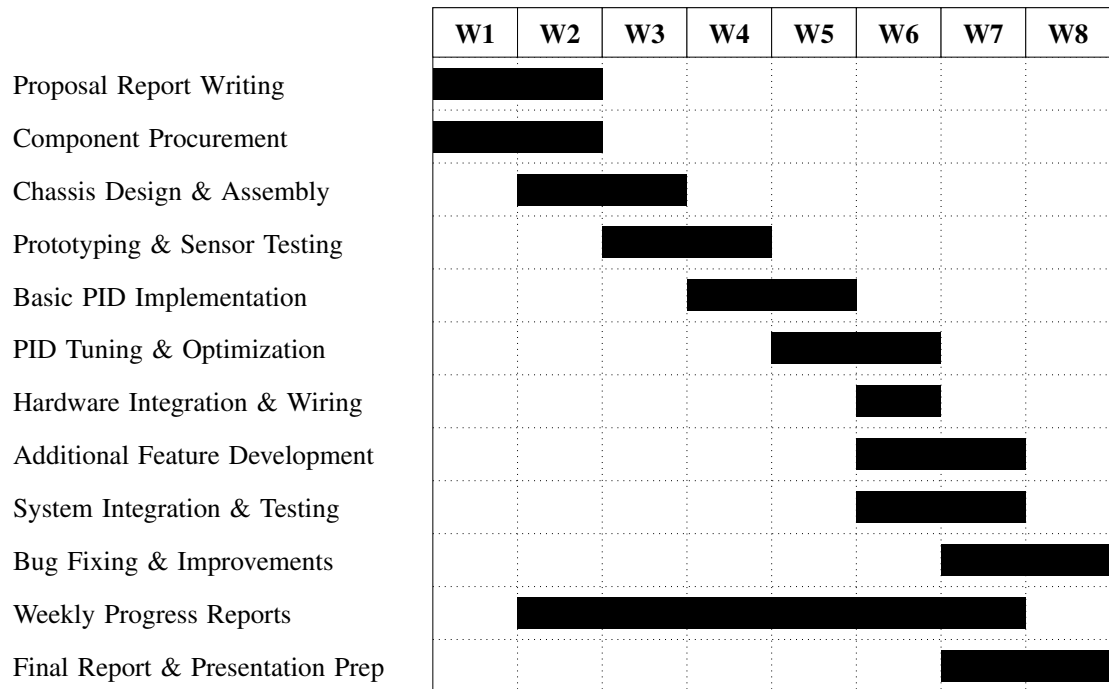
Table 4.1: Project Budget Estimation

S.N	Component	Quantity	Rate (NPR)	Total (NPR)
1	MPU6050 (IMU Sensor)	1	800	800
2	Microcontroller (ESP32/Arduino)	1	900	900
3	Motor Driver (TB6612FNG)	1	500	500
4	DC Gear Motors (12V)	2	700	1400
5	Li-ion Battery (11.1V)	1	1200	1200
6	Buck Converter	1	300	300
7	Wheels	2	250	500
8	Chassis	1	1000	1000
9	Jumper Wires (Set)	2	150	300
10	Miscellaneous	1	1000	1000
Total Cost				7900

CHAPTER 5

TIME SCHEDULE

Project Duration: 8 Weeks (2 Months)



REFERENCES

- [1] H. Zhang and N. Mohamad Nor, “Control strategies for two-wheeled self-balancing robotic systems: A comprehensive review,” *Robotics*, vol. 14, no. 8, p. 101, 2025.
- [2] R. A. D. D. Dharmasiri and M. K. Jayananda, “Evaluation of the dynamic performance and practical limitations of a two-wheeled self-balancing robot,” *International Journal of Electrical and Computer Engineering*, vol. 15, no. 4, pp. 3613–3620, 2025.
- [3] A. Sharma, R. Singh, V. Kumar, and M. Rahman, “Self-balancing mobile robot with bluetooth control: Design, implementation, and performance analysis,” *Automation*, vol. 6, no. 3, p. 42, 2025.
- [4] L. Fernandez, M. Costa, and R. Silva, “Commercial hoverboard reverse engineering and repurposing for a stabilized platform: A recyclable solution for modular robotic bases,” *Sensors*, vol. 25, no. 12, p. 3833, 2025.
- [5] C.-T. Hsieh, “Development of proportional-integral-derivative based self-balancing robot using esp32 for stem education,” in *Engineering Proceedings*, vol. 92, no. 1. MDPI, 2025, p. 24.